

Program 3

MORE HINTS AND TIPS

First understand Unix File IO v. C++ stdio FileIO

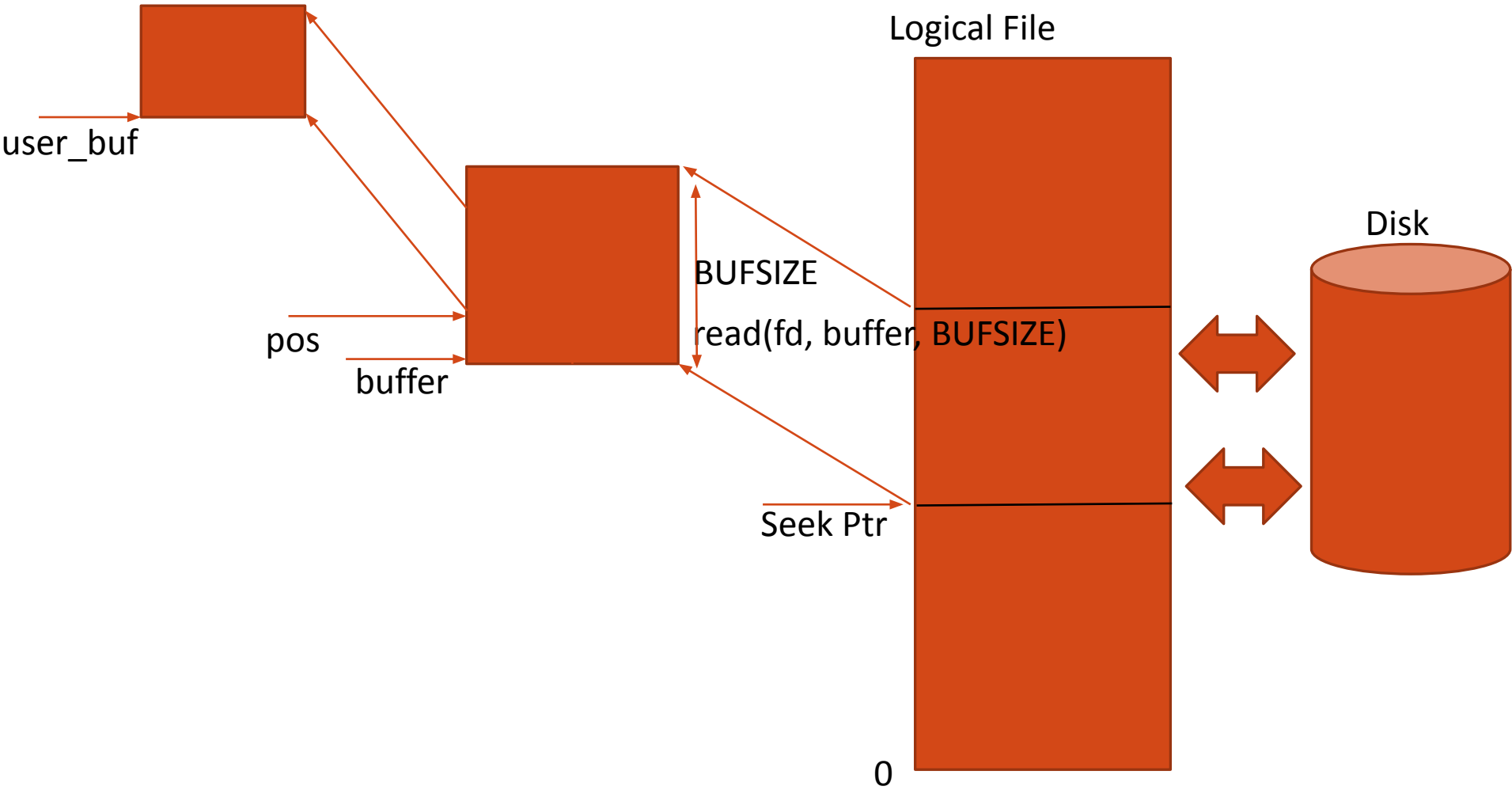
1. Understand and use File I/O using unix system calls: `open()`, `close()`, `seek()`, `read()`, `write()`
 2. Understand how file descriptors work
 3. Understand and use core functions File I/O using C++ Std IO Library
 1. `fopen()`, `fclose()`, `fseek()`, `fwrite()`, `fread()`
 2. `fgetc()`, `fputc()`, `fgets()`, `fputs()`
 4. Understand how a buffer is utilized in a File I/O C++ Std IO Library
 1. Use `fflush()`, `fpurge()`, `setbuf()`, `setvbuf()`
- Lab Three will help with this
 - Also, write code that uses all of the APIs above so that you know how they work.
 - You can also change the `#include "stdio.h"` to `#include <stdio.h>` and understand how the `driver.cpp` will behave

Understand the assignment

Now that you have a good understanding of the difference between unix IO and stdio.h file you should take some time to understand the assignment

1. Read the assignment slowly
2. Read the provided stdio.h and stdio_template.cpp
 1. Understand the data members of the FILE class
 2. Understand the fopen() code; read it line-by-line
3. Make sure you understand the differing buffering options

fread(user_buf, siz, 1, FILE)



Implement the non-buffered case first

The non-buffered case just translates fread calls into read calls.

By implementing these first you will have a feeling for how these things work.

Implement read functions first

- `fgetc()`
- `fgets()`
- `fread()`
- `fseek()`
- `fclose()`

Make sure you test and keep your tests running after each of these is implemented

Implement write functions

- `fputc()`
- `fputs()`
- `fwrite()`

Make sure you test and keep your tests running after each of these is implemented

Finish with the auxiliary functions
